

Microservice Architektur

Definition (nach Fowler):

Eine Anwendung besteht aus einer **Suite kleiner Services**, die:

- jeweils in eigenem Prozess laufen
 - über **leichte Kommunikationsmechanismen** (z.B. HTTP/REST) kommunizieren
 - unabhängig deploybar sind
 - eigene Datenhaltung besitzen
-

Grundprinzip

Microservices entstehen durch **funktionale Dekomposition**:

- Aufteilung nach **Business Funktionen**
- nicht nach technischen Schichten

Beispiel:

Monolith:

- Presentation
- Logic
- Data Access

Microservices:

- Order Service
 - Payment Service
 - User Service
-

Eigenschaften von Microservices

Jeder Microservice ist vollständig in Bezug auf:

- eigene Funktionalität
- eigenes Datenmanagement
- eigene Kommunikationsschnittstelle

Services sind klein genug für ein Team (ca. **4--9 Entwickler**).

Dezentralisiertes Datenmanagement

Wichtige Eigenschaft:

- Jeder Service besitzt seine **eigene Datenbank**

Vorteile:

- lose Kopplung
- polyglotte Persistenz (verschiedene Datenbanken möglich)
- unabhängige Skalierung

Beispiele:

- SQL
 - NoSQL
 - Sharding
-

Vorteile der Architektur

- unabhängige Weiterentwicklung von Services
- unabhängiges Deployment
- separate Skalierung einzelner Services
- unterschiedliche Technologien möglich

Software und Teams können **unabhängig skalieren**.

Dezentrale Governance

Standardisiert werden nur zentrale Aspekte:

- Kommunikationsprotokoll (HTTP)
- Datenformat (JSON)
- API Stil (REST)

Nicht standardisiert werden:

- Programmiersprachen
- Datenbanken

- Frameworks
-

Herausforderungen

Stabile Schnittstellen

Kommunikation meist über:

- HTTP / HTTPS
- REST APIs
- JSON

Alternativ:

- gRPC

APIs müssen **stabil bleiben**, damit Services unabhängig deploybar bleiben.

Versionierung von APIs

Möglichkeiten:

1. Version in URI
/v2/accounts
2. Custom Header
api-version: 2
3. Accept Header
Accept: application/vnd.accounts-v2+json

Empfehlung: Kombination aus **URI Version + Accept Header**.

Weitere Herausforderungen

- Systemtests schwieriger
- Authentifizierung über Gateways
- verteilte Transaktionen
- Logging über mehrere Services

Typische Lösung:

- zentrale Logging Systeme
 - Service Gateways
 - externe Koordinationsdienste
-

Microservice Infrastruktur

Microservices benötigen zusätzliche Infrastruktur:

- API Gateway
- Service Discovery
- Load Balancer
- Monitoring
- Logging
- Messaging Infrastruktur

Automatisierung über **DevOps Pipelines**.

API Gateway

API Gateway vereinfacht Clientzugriff.

Aufgaben:

- zentrale Adresse für Services
 - Routing von Requests
 - Load Balancing
 - Authentifizierung
 - Logging
-

Service Discovery

Problem:

- Services laufen auf dynamischen Instanzen.
- Clients dürfen keine festen IP-Adressen verwenden.

Lösung:

Service Registry / Discovery

Funktionen:

- Registrierung von Services
 - Finden verfügbarer Instanzen
 - Health Checks
 - Load Balancing
-

Beispiele für Discovery Systeme

- Consul
 - Netflix Eureka
 - etcd
-

Consul Beispiel

Consul bietet:

- Service Registry
 - verteilte Key-Value Datenbank
 - REST API
 - DNS API
 - Health Checks
-

Health Checks

Arten:

Dead Man Switch

Service sendet regelmäßig Heartbeat.

HTTP Check

Service wird über HTTP Endpoint geprüft.

Script Check

Script überprüft Zustand eines Services.

Koordinierungsdienste

Beispiel: **ZooKeeper**

Funktionen:

- Locks
- Warteschlangen
- Leader Election
- Gruppenmanagement
- Konfigurationsmanagement

Basierend auf Konsensus-Algorithmen.

Laufzeitprobleme

Microservices bestehen aus vielen Technologien:

- Web Frontend
- Datenbanken
- Message Queues
- Worker Prozesse

Problem:

- viele unterschiedliche Laufzeitumgebungen
 - schwierige Deploymentprozesse
-

Docker

Docker löst dieses Problem mit **Containern**.

Prinzip:

- Anwendung + Abhängigkeiten in Container verpacken
- Container läuft identisch auf allen Systemen

Vorteile:

- portabel
 - reproduzierbar
 - schnelle Deploymentprozesse
-

Docker basiert auf Linux Funktionen

- Namespaces
- cgroups
- SELinux

Diese ermöglichen isolierte Laufzeitumgebungen.

Docker Workflow

1. Dockerfile erstellen
2. Image bauen
3. Image in Registry pushen
4. Container aus Image starten

Typische Befehle:

- docker build
 - docker pull
 - docker run
 - docker push
-

Docker Image Struktur

Ein Image besteht aus Schichten:

- Base Image
- Libraries
- Application Server
- Anwendungscode

Images sind **read-only Layer**.

Kubernetes

Kubernetes ist ein **Container-Orchestrierungssystem**.

Aufgaben:

- Verwaltung von Container Clustern

- Deployment von Anwendungen
 - Skalierung
 - Monitoring
 - automatische Neustarts
-

Pods

Pod = kleinste Ausführungseinheit.

Eigenschaften:

- Gruppe von Containern
 - teilen sich Netzwerk und Storage
 - laufen auf gleichem Host
 - sind **ephemeral** (flüchtig)
-

Kubernetes Networking

Regeln:

- jeder Pod besitzt eigene IP
 - Pods können direkt miteinander kommunizieren
 - keine NAT erforderlich
-

CNI (Container Network Interface)

Standard für Container-Netzwerke.

Plugins:

- Calico
 - Weave
 - Flannel
-

Workload Objekte

Kubernetes nutzt **Workload Objekte** zur Verwaltung von Pods.

Beispiele:

- ReplicaSet
 - Deployment
 - StatefulSet
-

ReplicaSet

Stellt sicher, dass eine bestimmte Anzahl von Pods läuft.

Parameter:

- replicas
- selector
- template

Controller überwacht Pods und erstellt neue bei Ausfall.

Kubernetes Services

Services ermöglichen Zugriff auf Pods.

Eigenschaften:

- stabile IP-Adresse
- stabile DNS-Adresse
- dauerhaft im Cluster

Typen:

- ClusterIP
 - NodePort
 - LoadBalancer
 - ExternalName
-

LoadBalancer Service

Erstellt externe IP-Adresse.

Funktion:

- verteilt Traffic auf mehrere Pods
 - Integration mit Cloud Load Balancer
-

Ingress

Ingress routet externen Traffic auf interne Services.

Funktionen:

- Routing
- Load Balancing
- TLS Termination

Implementiert durch **Ingress Controller**.

Blue-Green Routing

Deployment Strategie:

- alte Version (Blue)
- neue Version (Green)

Traffic wird schrittweise umgeleitet.

Beispiel:

50% Traffic auf neue Version.

Observability

Wichtige Aspekte:

- Logging
- Monitoring
- Tracing

Werkzeuge:

- Prometheus
- Grafana
- ELK Stack

Fazit

Microservice Architekturen benötigen:

- Container Technologien (Docker)
- Orchestrierung (Kubernetes)
- zusätzliche Infrastruktur (Gateway, Discovery)

Diese bilden ein **Microservice Laufzeit-Ökosystem**.